Software Developer Coop Sections Exam

คุณคือ Software Developer ที่เพิ่งได้รับ Ticket จาก Tech Lead
บริษัทขนส่งโลจิสติกส์กำลังสร้าง **Fleet Management Platform** สำหรับจัดการยานพาหนะ ติดตามเส้นทาง และวางแผนการบำรุงรักษา Tech Lead ส่ง ticket มาสั้น ๆ ว่า

> “เราต้องการ dashboard ที่ dispatcher นั่งดูได้ตลอดวัน — เห็นว่ารถอยู่ที่ไหน วิ่งเส้นทางไหน และคันไหนใกล้ถึงรอบซ่อม ทำให้รันได้ก่อน เดี๋ยวค่อย scale — มีเวลา 3 ชั่วโมง"

requirement ไม่ครบ — นั่นคือเจตนา คุณต้องตัดสินใจเองว่าจะ implement อะไร ลำดับไหน และ trade-off อะไรที่ยอมรับได้

Tech stack ไม่จำกัด — เลือกแล้วต้องบอกใน README ว่าทำไมถึงเลือกนั้น ใช้ AI ได้ทุกอย่าง

Data Structure
Vehicle

```
{
  "id": "veh_001",
  "license_plate": "กข-1234",
  "type": "TRUCK | VAN | MOTORCYCLE | PICKUP",
  "status": "ACTIVE | IDLE | MAINTENANCE | RETIRED",
  "driver_id": "drv_001",
  "context": {
    "brand": "Isuzu",
    "model": "D-Max",
    "year": 2020,
    "fuel_type": "DIESEL | GASOLINE | ELECTRIC | HYBRID",
    "mileage_km": 45230,
    "last_service_km": 40000,
    "next_service_km": 50000
  }
}
```

Trip

```
{
  "id": "trp_001",
  "vehicle_id": "veh_001",
  "driver_id": "drv_001",
  "status": "SCHEDULED | IN_PROGRESS | COMPLETED | CANCELLE
D",
  "started_at": "2024-01-15T08:00:00Z",
  "ended_at": null,
  "context": {
    "origin": "กรุงเทพฯ",
    "destination": "เชียงใหม่",
    "distance_km": 696,
    "cargo_type": "GENERAL | FRAGILE | HAZARDOUS | REFRIGER
ATED",
    "cargo_weight_kg": 1500,
    "related_events": {
      "parent_id": null,
      "checkpoint_ids": ["chk_001", "chk_002", "chk_003"]
    }
  }
}
```

Checkpoint

```
{
  "id": "chk_001",
  "trip_id": "trp_001",
  "sequence": 1,
  "status": "PENDING | ARRIVED | DEPARTED | SKIPPED",
  "arrived_at": "2024-01-15T10:30:00Z",
  "context": {
    "location_name": "นครสวรรค์",
    "latitude": 15.7047,
    "longitude": 100.1372,
    "purpose": "FUEL | REST | DELIVERY | PICKUP | INSPECTIO
N",
    "notes": "ส่งของบางส่วน"
```

```
  }
}
```

Maintenance

```
{
  "id": "mnt_001",
  "vehicle_id": "veh_001",
  "status": "SCHEDULED | IN_PROGRESS | COMPLETED | OVERDU
E",
  "scheduled_at": "2024-02-01T09:00:00Z",
  "context": {
    "type": "OIL_CHANGE | TIRE | BRAKE | ENGINE | INSPECTIO
N | REPAIR",
    "mileage_at_service": 50000,
    "technician": "ช่างสมชาย",
    "cost_thb": null,
    "notes": "เปลี่ยนถ่ายน้ำมันเครื่อง",
    "related_events": {
      "parent_id": null,
      "parts_replaced": ["filter_001", "oil_001"]
    }
  }
}
```

ข้อ 1 Authentication

1.1 จงเขียน `POST /auth/login` ที่รับ username และ password, เปรียบเทียบกับ DB โดย password ต้อง hash ด้วย bcrypt และ return JWT access token อายุ 15 นาที กับ refresh token อายุ 7 วัน ถ้า credentials ผิดต้องตอบ 401 พร้อม error schema ที่กำหนด

1.2 จงเขียน JWT middleware ที่ protect ทุก route และแยก permission ตาม role ได้ดังนี้ — `DISPATCHER` จัดการ trips และ checkpoints ได้ แต่ลบ vehicle ไม่ได้, `ADMIN` ทำได้ทุกอย่าง ถ้า token หมดอายุต้องตอบ 401 พร้อมบอกให้ refresh

1.3 จงเขียน `POST /auth/refresh` ที่รับ refresh token และ return access token ใหม่ ถ้า refresh token หมดอายุหรือไม่ถูกต้องต้องตอบ 401 และ force logout

1.4 จงเขียน login form ใน FE ที่ส่ง credentials ไป BE จริง, เก็บ JWT ใน memory หรือ httpOnly cookie, auto refresh เมื่อ access token หมดอายุโดยไม่ให้ user รู้สึก และ

redirect ไปหน้า login พร้อม message เมื่อ refresh token หมดอายุ

ข้อ 2 Vehicle & Driver Management

2.1 จงเขียน `POST /vehicles` และ `POST /drivers` ที่บันทึกลง MySQL พร้อม input validation ครบทุก field และ return error schema `{ "error": { "code": "...", "message": "...", "details": {} } }` ทุก case

2.2 จงเขียน business logic ดังนี้ในครั้งเดียว — vehicle ที่มี trip `IN_PROGRESS` อยู่ลบไม่ได้, driver ที่ license หมดอายุ assign ให้รถใหม่ไม่ได้, เมื่อ `mileage_km` เกิน `next_service_km` ต้องเปลี่ยน status เป็น `MAINTENANCE` อัตโนมัติและสร้าง maintenance record `SCHEDULED` ทันทีใน **transaction เดียวกัน**

2.3 จงเขียน `GET /vehicles/:id/history` ที่ return trips และ maintenance ทั้งหมดรวมกันใน array เดียว เรียงตาม date โดยแต่ละ item มี `type` บอกว่าเป็น `trip` หรือ `maintenance`

2.4 จงสร้าง vehicle list ใน FE จาก API จริง พร้อม filter ที่ทำงานบน client-side ใน 100ms — filter ได้ตาม status, type, driver, free-text search และ URL ต้อง reflect ทุก filter state เพื่อ restore ได้เมื่อ reload

2.5 จงเขียน vehicle status transition ใน FE ที่ block การเปลี่ยน status ที่ไม่ถูกต้อง — `IDLE → ACTIVE → IDLE`, `ACTIVE → MAINTENANCE → IDLE` ถ้า transition ผิดต้องแสดง error message ที่บอกว่า transition ที่ทำได้ตอนนี้คืออะไร

ข้อ 3 Trip Management

3.1 จงเขียน `POST /trips` ที่ตรวจสอบว่า vehicle 1 คันมีได้แค่ 1 trip `IN_PROGRESS` ในเวลาเดียวกัน และ `PATCH /trips/:id/complete` ที่เมื่อ `COMPLETED` ต้องอัปเดต `mileage_km` ด้วย `distance_km` ใน **transaction เดียวกัน** และตรวจสอบ `next_service_km` ทันทีหลัง update

3.2 จงเขียน `PATCH /checkpoints/:id/status` ที่ update status ของ checkpoint โดย `ARRIVED` ต้องก่อน `DEPARTED` และต้องทำตาม sequence ข้ามไม่ได้ ถ้าผิด sequence ต้องตอบ error พร้อมบอกว่า checkpoint ก่อนหน้ายังไม่เสร็จ

3.3 จงสร้าง multi-step trip form ใน FE 3 steps — (1) เลือก vehicle + driver, (2) กำหนด origin, destination, cargo, (3) เพิ่ม checkpoints อย่างน้อย 1 จุด โดยแต่ละ step validate ก่อน proceed ได้ และกด back ต้องไม่สูญหายข้อมูลที่กรอกไปแล้ว

3.4 จงเขียน trip tracker ใน FE ที่แสดง checkpoint progress เป็น visual step indicator — แต่ละ step แสดง location_name, purpose, status พร้อม color coding และ update status ได้พร้อม optimistic update ที่ simulate delay 300–800ms แบบ random, fail 30%, rollback state กลับเมื่อ fail พร้อม error message ที่อ่านแล้วรู้ว่าต้องทำอะไรต่อ

ข้อ 4 Maintenance & Alert Engine

4.1 จงออกแบบและเขียน alert engine ที่ trigger alerts อัตโนมัติโดยมี rules ดังนี้ — `mileage_km` เกิน `next_service_km` → "Vehicle Due for Service", maintenance `SCHEDULED` เลยมาแล้วเกิน 3 วัน → "Overdue Maintenance", driver license หมดอายุภายใน 30 วัน → "License Expiring Soon", trip `IN_PROGRESS` นานเกิน estimated duration 150% → "Trip Delayed" โดย **rule ใหม่ต้องเพิ่มได้โดยไม่แก้ core engine code เลยแม้แต่บรรทัดเดียว**

4.2 จงเขียน `GET /alerts` ที่ return alerts ทั้งหมดพร้อม `severity` (WARNING / CRITICAL), `affected_resource_type`, `affected_resource_id` และ `message` filter ได้ตาม severity และ resource_type

4.3 จงสร้าง maintenance schedule ใน FE ที่ highlight `OVERDUE` สีแดง และ due ภายใน 7 วัน สีเหลือง โดยคำนวณจาก `scheduled_at` เทียบกับวันปัจจุบัน และแสดง alerts จาก `GET /alerts` ใน panel แยกต่างหาก

ข้อ 5 Dashboard & Audit Log

5.1 จงสร้าง summary metric cards ใน FE ที่ดึงข้อมูลจาก BE จริง แสดง total vehicles, active trips today, total distance today, maintenance overdue count โดยแต่ละ card กดแล้ว navigate ไปหน้าที่ filter ตรงกันได้ทันที

5.2 จงสร้าง chart อย่างน้อย 2 แบบโดยใช้ library จริง — vehicles by status (pie หรือ donut) และ trip distance trend 7 วันล่าสุด (line หรือ bar)

5.3 จงเขียน audit log middleware ใน BE ที่บันทึกทุก action สำคัญลงตาราง `audit_logs` ได้แก่ login สำเร็จและล้มเหลว, สร้าง/แก้/ลบ vehicle, assign driver, เปลี่ยน trip status, update checkpoint โดย audit log ต้อง append-only ห้ามมี endpoint ที่แก้ไขหรือลบ records ได้

5.4 จงเขียน `GET /audit-logs` ที่ filter ได้ตาม user_id, action, resource_type และ date range โดย DISPATCHER เห็นได้เฉพาะ log ของตัวเอง ADMIN เห็นได้ทุก log

---

Constraints ที่ห้ามละเมิด

- ต้องต่อ MySQL จริง
- Alert engine ต้องเพิ่ม rule ใหม่ได้โดยไม่แก้ existing code
- Password ต้อง hash ด้วย bcrypt ห้าม store plaintext
- JWT ต้องมี expiry และ refresh mechanism
- Error response ต้องมี **Pattern** เดียวกันทุก endpoint

- mileage update และ maintenance auto-create ต้องอยู่ใน transaction เดียวกัน
- Filter ต้อง reflect ใน URL และ restore ได้เมื่อ reload
- Vehicle status transition ผิดต้อง block พร้อม error message
- Trip progress ต้องเป็น visual ไม่ใช่แค่ text